

Documento de Arquitetura do Sistema CliVet

Caroline Feitosa dos Santos¹, Marcos Vinícius Carvalho Silva¹

¹Instituto Federal de Educação, Ciência e Tecnologia da Bahia (IFBA)
Campus Vitória da Conquista

202221190031@ifba.edu.br, 202311190005@ifba.edu.br

Resumo. Este documento apresenta a definição arquitetural do sistema de gerenciamento CliVet, desenvolvido para atender às necessidades operacionais da clínica veterinária. A arquitetura foi elaborada com base no documento de requisitos do sistema, considerando aspectos relacionados ao gerenciamento de clientes, produtos, medicamentos, estoque e vendas. Além disso, são registradas as principais decisões arquiteturais por meio de Architecture Decision Records (ADRs).

1. Introdução

Este trabalho apresenta a definição arquitetural do sistema CliVet, uma aplicação web voltada ao gerenciamento administrativo da clínica veterinária. A arquitetura foi elaborada a partir dos requisitos funcionais e não funcionais presentes no documento de especificação de requisitos.

2. Resumo dos Requisitos

Foram selecionados os requisitos funcionais de maior impacto arquitetural, por influenciarem diretamente as decisões de estruturação, validação de dados e integridade referencial do banco de dados. Os principais requisitos funcionais do sistema incluem:

- [RF001]: Cadastrar Cliente;
- [RF005]: Cadastrar Produtos;
- [RF009]: Cadastrar Medicamentos;
- [RF013]: Cadastrar no Estoque;
- [RF017]: Cadastrar Venda;
- [RF003], [RF007], [RF011], [RF015], [RF018]: Pesquisa em módulos;
- [RF004], [RF008], [RF012], [RF016], [RF020]: Exclusão com confirmação.

Os principais requisitos não funcionais incluem¹:

- NF001: Interface amigável;
- NF002: Componentes WEB;
- NF003: Utilização de banco de dados;
- NF005: Agilidade na execução das operações.

¹Os requisitos não funcionais apresentados são genéricos, pois o documento de requisitos foi desenvolvido em outra disciplina e foi disponibilizado dessa forma.

3. Proposta Arquitetural

A arquitetura adotada, exibida na Figura 1, segue o estilo arquitetural, MVC (*Model-View-Controller*), em conjunto com uma arquitetura em camadas. Essa abordagem foi escolhida devido à necessidade de separação entre regras de negócio, interface do usuário e persistência de dados, facilitando manutenção, evolução e organização do sistema. Os principais componentes da arquitetura são:

- Interface WEB (*View*): responsável pela interação com os usuários do sistema;
- Camada de Controle (*Controller*): responsável pelo processamento das requisições e controle do fluxo do sistema;
- Camada de Negócio (*Model/Service*): responsável pelas regras de negócio relacionadas a clientes, produtos, medicamentos, estoque e vendas;
- Banco de Dados: responsável pelo armazenamento persistente das informações;
- Sistema de Relatórios: responsável pela geração de relatórios administrativos e operacionais.

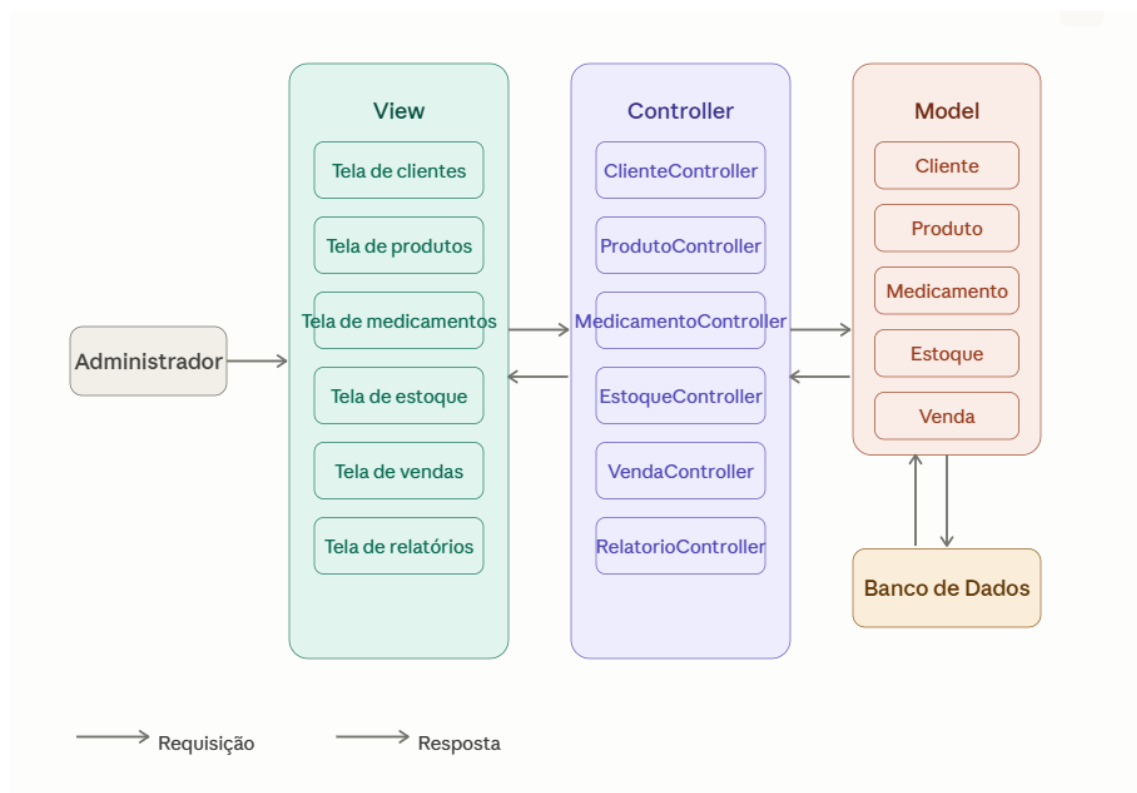


Figura 1. Visão geral da arquitetura do sistema

4. Decisões Arquiteturais

Agora, considerando os requisitos apresentados na Seção 2, abordamos as decisões arquiteturais que devem ser implementadas. Cada uma das sub-seções seguintes trata de um decisão (uma decisão por cada ADR).

4.1. ADR-01: Utilização da Arquitetura MVC

Contexto:

O sistema possui múltiplos módulos administrativos e necessita de organização estrutural para facilitar manutenção e evolução futura.

Decisão:

Adotar o padrão arquitetural MVC (*Model-View-Controller*).

Status: Aceito

Consequências:

- Separação clara entre interface, regras de negócio e persistência;
- Facilidade de manutenção e escalabilidade;
- Maior complexidade estrutural inicial.

4.2. ADR-02: Utilização de Banco de Dados Relacional

Contexto:

O sistema manipula dados estruturados relacionados a clientes, produtos, estoque, medicamentos e vendas.

Decisão:

Utilizar banco de dados relacional MySQL para persistência das informações.

Status: Aceito

Consequências:

- Maior integridade e consistência dos dados;
- Facilidade para consultas relacionais;
- Necessidade de modelagem mais rigorosa.

4.3. ADR-03: Desenvolvimento de Interface WEB

Contexto:

O requisito NF002 determina a utilização de componentes WEB e facilidade de uso.

Decisão:

Desenvolver o sistema como aplicação WEB responsiva.

Status: Aceito

Consequências:

- Facilidade de acesso em diferentes dispositivos;
- Não necessita instalação local;
- Dependência de um servidor acessível na rede local ou via internet para funcionamento do sistema.

4.4. ADR-04: Implementação de Controle de Estoque com Validade

Contexto:

O sistema necessita controlar validade de produtos e medicamentos presentes no estoque.

Decisão:

Implementar módulo de estoque integrado aos módulos de produtos e medicamentos.

Status: Aceito

Consequências:

- Melhor controle operacional;
- Redução de perdas por vencimento;
- Maior complexidade na lógica de atualização de estoque.

4.5. ADR-05: Implementação de Mecanismos de Pesquisa

Contexto:

Diversos requisitos funcionais exigem pesquisa rápida de informações cadastradas.

Decisão:

Implementar mecanismos de busca em clientes, produtos, medicamentos, estoque e vendas.

Status: Aceito

Consequências:

- Maior agilidade operacional;
- Melhor experiência do usuário;
- Necessidade de otimização de consultas no banco de dados.

4.6. ADR-06: Centralização das Regras de Negócio

Contexto:

O sistema possui diversas validações relacionadas a cadastro, edição e exclusão de informações.

Decisão:

Centralizar regras de negócio na camada de serviços da aplicação.

Status: Aceito

Consequências:

- Redução de duplicidade de código;
- Maior organização do sistema;
- Maior complexidade arquitetural, exigindo maior esforço de abstração e desenvolvimento inicial.

4.7. ADR-07: Priorização de Desempenho nas Operações

Contexto:

O requisito NF005 exige agilidade nas operações do sistema.

Decisão:

Otimizar consultas ao banco de dados e utilização de paginação nas listagens.

Status: Aceito

Consequências:

- Melhor desempenho do sistema;
- Redução de consumo de recursos;
- Maior complexidade de implementação.

5. Uso de Ferramentas de Apoio

- Código-fonte das ferramentas MCP: [repositório no GitHub](#)
- Vídeo de demonstração das ferramentas no YouTube: [vídeo demonstrativo](#)

6. Análise Crítica

A arquitetura proposta atende aos principais requisitos do sistema. Entretanto, algumas limitações podem surgir, como dependência de conexão, complexidade do sistema adicionando novos módulos e otimização contínua do banco de dados.

Os principais *trade-offs* identificados incluem:

- Organização arquitetural vs. complexidade inicial;
- Desempenho vs. maior processamento de validações;
- Flexibilidade do sistema vs. custo de manutenção.

Melhorias futuras podem incluir:

- Sistema de autenticação com múltiplos perfis;
- Geração avançada de relatórios;
- Integração com sistemas de pagamento;
- Sistema automático de alertas de validade.

7. Conclusão

A arquitetura definida para o sistema CliVet atende aos requisitos propostos, equilibrando organização estrutural e simplicidade operacional. O uso de decisões arquiteturais explícitas no formato ADR permite maior clareza e rastreabilidade das escolhas realizadas, facilitando a manutenção e futuras evoluções do sistema.